

From Omission Fault Tolerant
To Byzantine Fault Tolerant

Plan for today

Consensus

Partial Synchrony

Primary-Backup replication paradigm (variant of Paxos)

From Primary-Backup with Omission faults to Byzantine faults

State Machine Replication (SMR)

Consensus

A voting protocol for making decisions

Players have initial input (*authenticated* for our setting)

All honest players must eventually *decide* on a value (*Termination*)

No two honest players commit to different values (*Agreement*)

Add adversary (Omission, Byzantine)

Networking Model (Synchrony, Asynchronous, Partial Synchrony)

Trivial!

Always decide nothing (*nil* value)

If all the honest nodes have the same input, then this must be the decision value (*Validity*)

Consensus in the Client-Server Model

Clients only talk to servers

Clients send authenticated requests; they may be omission faulty

Servers commit to a request

Clients can learn which request was committed

Can be used to create a sequence of committed commands

Focus on an abstract single-shot instance

- *Termination*: all honest servers terminate and *decide* on a value
- *Agreement*: all honest servers *decide* on the same authenticated input
- *Validity*: if all honest servers start with the same input then this is the committed output

Partial Synchrony

Asynchrony: adversary can delay messages by any finite amount

Synchrony: adversary can delay messages by some known finite value Δ

Partial Synchrony:

- adversary can delay messages by any finite amount
- until some unknown finite point in time called Global Stabilization Time (GST)
- then adversary can delay messages by some known finite value Δ

Safety holds in asynchrony

Termination holds in synchrony

Extremely successful (Paxos, Raft, PBFT, Tendermint, HotStuff, ...)

Primary-Backup in the Omission Model

The omission model

- There are n replicas
- The adversary corrupts f replicas which can fail by not receiving or not sending each message

System works in *views*, in each view

- There is a designated Primary
- The rest are Backups

For simplicity: in view i the primary is $(i \bmod n)$

Primary-Backup in the Omission Model: Foundations

For simplicity we are going to assume $n = 2f+1$

The only math you will need:

- Quorum intersection (pigeonhole principle)
- Given a set of n elements: two subsets of $n-f$ elements must intersect at $n-2f$ elements
- For $n = 2f+1$, any two sets of $f+1$ must intersect at one element
- For $n = 3f+1$, any two sets of $2f+1$ must intersect at $f+1$ elements

Primary-Backup in the Omission Model: What Could Go Wrong?

Clients send request <append x>

Different replicas may hear different requests (or in different orders)

Primary chooses say the first client request <x>

- Commits x
- Sends <commit x> to all replicas

Primary chooses the first client request <x>

- Sends <propose x> to all replicas
- Commits x

Main challenge: the first primary may commit to x, but the next primary commits to x'

Primary-Backup in the Omission Model: View Change Protocol

Use a view change protocol to guarantee safety:

- Before a new primary starts, it runs a view change protocol
- If there is any possibility that some value was previously committed, the new primary must adopt that value

Three challenges:

1. Only commit to a value after you are sure later primaries can recover this value
2. Make the view change safe: only choose safe value to adopt
3. Make the view change live: don't get stuck waiting

Primary-Backup in the Omission Model: Algorithm Structure

1. Normal case protocol
 - allows the primary to commit
2. View change trigger protocol
 - a way to trigger the replacement of a primary
3. View change protocol
 - a way for a new primary to make safe choices

Primary-Backup in the Omission Model: Normal Case

1. Clients send requests <append x> to primary
 2. Primary (of view v) sends <propose x in view v> to all replicas
 3. Replicas send <ack x in view v> to Primary
 - Unless it has moved to a higher view
 4. Primary waits for **quorum ($n-f = f+1$)** of <ack x in view v> to **commit to x**
-
- Primary can tell clients and replicas of commit decision (pipeline)

Primary-Backup in the Omission Model: Stable Leader or Revolving

View change must be used for a failed primary

1. Stable leader:

- Primary sends <I am alive> heartbeat at intervals to all
- A replica that does not receive heartbeat from primary sends <I don't hear Primary, v > to all
- A replica hears $f+1$ <I don't hear Primary, v > or one <Primary down, v >
 - Move to view $v+1$: Stop responding to current Primary
 - Send <Primary down, v > to all

2. Revolving coordinator: do a view change every phase

- Even if there are no failures
- Use synchronized heartbeat mechanisms to have all replicas move to the same view
- Better fairness but more overhead in the normal case

View Change

Maybe the previous primary committed?

Maybe one of the previous primary committed?

New leader may need to adapt a value instead of choosing its own

Quorum intersection to the rescue:

- If some primary committed, then it used a write quorum ($f+1$)
- So reading from a quorum of $f+1$:
 - Is safe: primary will see this
 - Is live: can always be done

View Change: From view v to view $v+1$

New primary for view $v+1$:

- Send message **<view change for view $v+1$ > to all**
- A replica responds with <my maximal propose is x' at view v' , v >
 - Using the propose with maximal view v' it heard
 - Or send <null at view 0, v > if heard no propose

Primary waits for $n-f = f+1$ responses:

- Adopts the proposed value associated with the maximal view number, or
- Uses its own value if every message is <null at view 0, v >

From Omission to Byzantine

Two main problems:

- A faulty primary that equivocates - it sends different values to different replicas
- A faulty primary that adopts the wrong value (not the propose value with the highest view)

Solution approach:

- Add a sub-protocol to force primary to act like omission (no equivocation)
- Add a sub-protocol to guarantee that the primary will fail if using unsafe values

1. Using synchrony for safety (and quadratic communication)
 - Get synchronous protocol for $n = 2f+1$
2. Continue to not assume synchrony for safety (and maintain linear complexity)
 - Get partial synchrony protocol for $n = 3f+1$

Byzantine Model in Partial Synchrony

Partial Synchrony [Consensus in the Presence of Partial Synchrony]

- System is asynchronous till GST, synchronous after
- GST event must eventually happen

Authenticated Setting (*Public Key Infrastructure*)

- All messages sent from replicas are signed

Client requests are signed

Adversary controls corrupted nodes fully

Two primary attacks:

- Equivocate: tell different replicas different things
- Unsafe: adopt a non-safe value after view change

Byzantine Agreement in Partial Synchrony

Need $n \geq 3f+1$:

- Primary can equivocate
- Asynchronous split brain
- Can only wait for $n-f$ responses (termination and that Byzantine actors may never respond)
- To prevent being deceived, there must be an honest majority among the $n-f$ replicas
- Leads to $f < \frac{1}{2}(n-f)$, which simplifies to $f < n/3$

We assume $n = 3f+1$

Non-Equivocation (linear version)

Goal: given a (potentially) Byzantine primary, **transform** its send-to-all to a (potentially) omission fault primary send-to-all

1. Primary sends $\langle \text{pre-propose}, (\text{value}, v) \rangle$ to all
2. Replica sends $\langle \text{pre-propose ack}, (\text{value}, v) \rangle_i$ to primary for the first (value, v) it hears from primary
3. Primary gathers $n-f$ pre-propose acks with the same value and
 - a. Sends $\langle \text{propose}, (\text{value}, v), \text{proof} \rangle$ to all (where proof is $n-f$ signatures of pre-propose)
4. If a replica sees $\langle \text{propose}, (\text{value}, v), \text{proof} \rangle$ it **accepts** $\langle \text{propose}, (\text{value}, v) \rangle$ if
 - a. proof contains $n-f$ valid signatures for $\langle \text{pre-propose}, (\text{value}, v) \rangle$

Note: many protocols call it *pre-vote* and *pre-commit* instead of *pre-propose* and *propose*

Non-Equivocation

Claim: if a replica accepts $\langle \text{propose}, (\text{value}, v) \rangle$ then no replica will accept $\langle \text{propose}, (\text{value}', v) \rangle$ where $\text{value} \neq \text{value}'$

Proof by contradiction:

1. One replica sees $n-f$ $\langle \text{pre-propose} (\text{value}, v) \rangle$ and another sees $n-f$ $\langle \text{pre-propose} (\text{value}', v) \rangle$
2. The intersection is at least $f+1$, so at least one non-faulty in the intersection
3. Non-faulty will only send one pre-propose per view

Normal case in view v (one thing still missing)

1. Primary sends $\langle \text{pre-propose}, (\text{value}, v) \rangle$ to all
2. Replica i accepts $\langle \text{pre-propose}, (\text{value}, v) \rangle$ if
 - a. Not locked on higher view and not already cast a vote
 - b. Value is valid (signature from client) and primary is correct primary for view v
 - c. Sends: $\langle \text{pre-propose ack}, (\text{value}, v) \rangle_i$ to primary
3. Primary gathers $n-f = 2f+1$ pre-propose acks as a *propose-proof* (often called *Quorum Certificate*)
 - a. Sends $\langle \text{propose}, (\text{value}, v), \text{propose-proof} \rangle$ to all
4. Replica i accepts $\langle \text{propose}, (\text{value}, v), \text{proof} \rangle$ if
 - a. Not locked on higher view
 - b. propose-proof contains $n-f$ valid signatures of pre-propose
 - c. Updates its highest accepted propose view number to v**
 - d. Sends $\langle \text{propose ack}, (\text{value}, v) \rangle_i$
5. Primary gathers $n-f$ propose acks as a *decide-proof* to decide (value, v) (Primary can Commit)
 - a. Sends $\langle \text{Commit}, (\text{value}, v), \text{decide-proof} \rangle$ to all

View Change Trigger

1. If timeout(view v) then
 - a. send $\langle \text{complain } v \rangle$ and start timer for view $v+1$
2. If Primary of view $v+1$ sees $f+1$ $\langle \text{complain } v \rangle$ then
 - a. start view change $v+1$
3. Primary uses the $f+1$ complain messages as a *view-change-proof*
4. (If you see *view-change-proof* for w , then update timer for view w)

View Change

1. Primary sends <view change, $v+1$, view-change-proof> to all
2. Replica accepts if
 - a. *View-change-proof* is valid (contain $f+1$ complain messages)
 - b. Replica is not locked in a higher view
 - c. Stops responding to all lower views
 - d. responds to Primary with <VC ack $v+1$, (value, view), propose-proof>
 - i. (value, view) is the propose with the highest view that the replica heard (locked on)
 - ii. propose-proof is the $n-f$ pre-propose signatures for this propose
 - iii. Or empty if replica never accepted a propose message
3. Primary waits for (timeout and $n-f$) of <VC ack $v+1$ (*)> (Tendermint version)
 - a. adopts (value) of the ((value, w), propose-proof(w)) with the highest view that has a valid propose-proof
 - b. Sends <pre-propose, (value, $v+1$), propose-proof(w)> to normal case; or
 - c. Sends <pre-propose, (my-value, $v+1$), empty> if all VC ack messages were empty

Normal case in view v

1. Primary sends $\langle \text{pre-propose}, (\text{value}, v), \text{propose-proof}(w) \rangle$ to all
2. Replica i accepts $\langle \text{pre-propose}, (\text{value}, v), \text{propose-proof}(w) \rangle$ if
 - a. Not locked on higher view and not already cast a vote
 - b. Value is valid (signature from client or adopted and consistent with $\text{propose-proof}(w)$)
 - c. $\text{Propose-proof}(w)$ is valid and w is equal or larger than Replica's highest accepted propose view number
 - d. Sends: $\langle \text{pre-propose ack}, (\text{value } v) \rangle_i$ to primary
3. Primary gathers $n-f$ pre-propose acks as a *propose-proof*
 - a. Sends $\langle \text{propose}, (\text{value}, v), \text{propose-proof} \rangle$ to all
4. Replica i accepts $\langle \text{propose}, (\text{value}, v), \text{propose-proof} \rangle$ if
 - a. Not locked on higher view
 - b. propose-proof contains $n-f$ valid signatures of pre-propose
 - c. Updates its highest accepted propose view number to v
 - d. Sends $\langle \text{propose ack}, (\text{value}, v) \rangle_i$
5. Primary gathers $n-f$ propose acks as a *decide-proof* to decide (value, v)

Safety

Let v^* be the first view that any replica committed (value x , view v^*)

Prove by induction that any propose of view $v \geq v^*$ must be consistent with value x

Base case: show that in view v no value x' will be proposed

Induction argument: existence of a core of $f+1$ non-faulty that have a [highest accepted propose locked on value \$x\$](#)

Any later view will need to be consistent with their highest proposed values to progress

Liveness

If non-faulty primary is elected and the system is synchronous

Must wait for all honest parties to send view change ack

- Reason we use timeout and
- wait for $n-f$ view change ack messages

Missing even one honest can break liveness

Can we improve this?

Responsive: good primary progress is as fast as the network, does not depend on Δ (or any timeout)

We still need a timeout to handle a faulty primary

Responsiveness: Solution Approach

- Challenge:
 - one honest may have a propose and missing it can break liveness
- Recall
 - One honest may have a commit and missing it can break safety
- We solved that by adding a round so that
 - If one honest commits, then at least $f+1$ honest have a propose
 - So view change will adopt correctly, and we will be safe
- Solve the liveness problem similarly:
 - If one honest accept a propose, then $f+1$ honest have a key

HotStuff: Normal case in view v

1. Primary sends $\langle \text{pre-propose}, (\text{value}, v), \text{key-proof}(w) \rangle$ to all
2. Replica i accepts $\langle \text{pre-propose}, (\text{value}, v), \text{key-proof}(w) \rangle$ if
 - a. Not locked on higher view and not already cast a vote
 - b. Value is valid (signature from client *or adopted and consistent with* $\text{key-proof}(w)$)
 - c. $\text{key-proof}(w)$ is valid and w is equal or larger than Replica's highest accepted propose view number
 - d. Sends $\langle \text{pre-propose ack}, (\text{value}, v) \rangle_i$ to primary
3. Primary gathers $n-f$ pre-propose acks as a *key-proof*
 - a. Sends $\langle \text{key}, (\text{value}, v), \text{proof} \rangle$ to all
4. Replica i accepts $\langle \text{key}, (\text{value}, v), \text{proof} \rangle$ if not locked on higher view and key-proof contains $n-f$ valid signatures of pre-propose ack
 - a. *Updates its highest accepted key view number to v*
 - b. Sends $\langle \text{key ack}, (\text{value}, v) \rangle_i$
5. Primary gathers $n-f$ *key acks* as a *propose-proof*
 - a. Sends $\langle \text{propose}, (\text{value}, v), \text{propose-proof} \rangle$ to all
6. Replica i accepts $\langle \text{propose}, (\text{value}, v), \text{proof} \rangle$ if not locked on higher view and propose-proof contains $n-f$ valid signatures of key ack
 - a. *Updates its highest accepted propose view number to v*
 - b. Sends $\langle \text{propose ack}, (\text{value}, v) \rangle_i$
7. Primary gathers $n-f$ propose acks as a *decide-proof* to decide (value, v)

HotStuff: View Change

1. Primary sends <view change, $v+1$, view-change-proof> to all
2. Replica accepts if
 - a. *view-change-proof* is valid (contain $f+1$ complain messages)
 - b. Replica is not locked in a higher view
 - c. stops responding to all lower views
 - d. Responds to Primary with <VC ack, $v+1$, (value, view), *key-proof*>
 - i. (value, view) is the *key* with the highest view that the replica heard
 - ii. *key-proof* is the $n-f$ key signatures for this propose
 - iii. Or empty if replica never accepted a key message
3. Primary waits for $n-f$ <VC ack, $v+1$, (*)>
 - a. adopts (value) of the ((value, w), *key-proof*(w)) with the highest view that has a valid *key-proof*
 - b. Sends <pre-propose, (value, $v+1$), *key-proof*(w)> to normal case; or
 - c. Sends <pre-propose, (my-value, $v+1$), empty> if all VC ack messages were empty

Reducing Latency and Correct View Change

- HotStuff Latency is 3 round trips (6 network delays)
- Byzantine Primary-Backup (Tendermint version) is 2 round trips (3 or 4 network delays by using all-to-all)
- Can we do 1 round trip?
 - In the best case, yes!
 - Not as easy as it seems... (fast and slow path)
- Optimism wins over pessimism
 - If you expect everyone to be online and honest you can get better performance
- Pessimism wins over optimism
 - If you are constantly under attack

BFT Protocols

Practical Byzantine Fault Tolerance (PBFT)

Tendermint

Casper The Friendly Finality Gadget (FFG)

HotStuff

The Ideal State Machine Paradigm

- State machine:
 - Deterministic function from old state and command to new state
 - Examples: key-value store, virtual machine, etc
- Suggested by Lamport
- Replace an ideal functionality with a robust implementation
- Goal:
 - Safety
 - Liveness

From Single Shot Consensus to State Machine Replication

- Two main basic tasks:
 - a. Commit to a sequence of commands
 - b. Implement a State Machine given a sequence of commands
- The power of a state machine:
 - a. State machine virtualization: reconfigure the state machine as a state machine command
 - b. Reconfigure the replication set as a state machine command
 - c. Reconfigure the replication protocol as a state machine command

From Single Shot Consensus to State Machine Replication

1. Commit to a sequence (log) of commands
 2. Implement a State Machine given a sequence of commands
- From single shot to a sequence:
 - Start committing on a slot $i+1$ only after all slots i and below are committed
 - Maintain a window of α open slots. Start working on a slot $i + \alpha + 1$ only after all slots i and below are committed
 - Concurrently work on multiple slots
 - Use pipelining techniques to save communication